Symbolic Execution

Finding bugs

Cristina David

Lecture 12

COMSM0067 — Advanced Topics in Programming Languages
University of Bristol

A change of goal

Lectures 8–11: prove $\{P\}\ c\ \{Q\}$ for *every* input.

A change of goal

Lectures 8–11: prove $\{P\}\ c\ \{Q\}$ for *every* input.

Today: find one input that breaks it.

```
if x < 0 then y := -x else y := x;
assert y > 0
```

A change of goal

Lectures 8–11: prove $\{P\}\ c\ \{Q\}$ for *every* input.

Today: find one input that breaks it.

```
if x < 0 then y := -x else y := x;
assert y > 0
```

Is there an input that makes the assertion fail?

Two ways to compute program states

A proof, and an alarm

$$\mathsf{wp}(\text{if } b \text{ then } c_1 \text{ else } c_2,\ Q) \ = \ (b \Rightarrow \mathsf{wp}(c_1, Q)) \ \wedge \ (\neg b \Rightarrow \mathsf{wp}(c_2, Q))$$

```
if x > 0 then y := 1
      else y := 0;
```

$\{\, Q \,\}$

```
if x > 0 then z := 10 / y
```

Take $Q \equiv 0 \leq y \leq 1$.

A proof, and an alarm

$$\text{wp}(\text{if } b \text{ then } c_1 \text{ else } c_2,\ Q) \;=\; (b \Rightarrow \text{wp}(c_1, Q)) \;\wedge\; (\neg b \Rightarrow \text{wp}(c_2, Q))$$

```
if x > 0 then y := 1
      else y := 0;
```

$\{\, Q \,\}$

```
if x > 0 then z := 10 / y
```

Take $Q \equiv 0 \leq y \leq 1$. Push it up through each branch:

$$\text{wp}(y := 1,\ Q) = 0 \leq 1 \leq 1 \;\equiv\; \text{true}$$
$$\text{wp}(y := 0,\ Q) = 0 \leq 0 \leq 1 \;\equiv\; \text{true}$$

A proof, and an alarm

$$\text{wp}(\text{if } b \text{ then } c_1 \text{ else } c_2,\ Q) \;=\; (b \Rightarrow \text{wp}(c_1, Q)) \;\wedge\; (\neg b \Rightarrow \text{wp}(c_2, Q))$$

```
if x > 0 then y := 1
      else y := 0;
```

$\{\, Q \,\}$

```
if x > 0 then z := 10 / y
```

Take $Q \equiv 0 \leq y \leq 1$. Push it up through each branch:

$$\text{wp}(y := 1,\ Q) = 0 \leq 1 \leq 1 \;\equiv\; \text{true}$$
$$\text{wp}(y := 0,\ Q) = 0 \leq 0 \leq 1 \;\equiv\; \text{true}$$

$$\text{wp}(\texttt{if } \ldots,\ Q) = (x > 0 \Rightarrow \text{true}) \wedge (x \leq 0 \Rightarrow \text{true})$$
$$\equiv \text{true}\ \checkmark$$

A proof, and an alarm

$$\text{wp}(\text{if } b \text{ then } c_1 \text{ else } c_2,\ Q) \;=\; (b \Rightarrow \text{wp}(c_1, Q)) \;\wedge\; (\neg b \Rightarrow \text{wp}(c_2, Q))$$

```
if x > 0 then y := 1
      else y := 0;
```

$\{\, Q \,\}$

```
if x > 0 then z := 10 / y
```

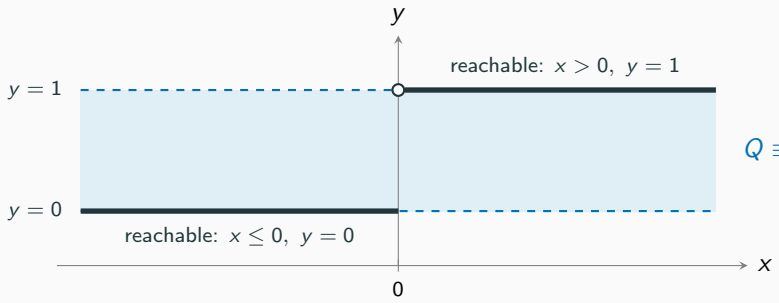
Take $Q \equiv 0 \le y \le 1$. Push it up through each branch:

$$\text{wp}(y := 1,\ Q) = 0 \le 1 \le 1 \;\equiv\; \text{true}$$
$$\text{wp}(y := 0,\ Q) = 0 \le 0 \le 1 \;\equiv\; \text{true}$$

$$\text{wp}(\texttt{if } \ldots,\ Q) = (x > 0 \Rightarrow \text{true}) \wedge (x \le 0 \Rightarrow \text{true})$$
$$\equiv \text{true}\ \checkmark$$

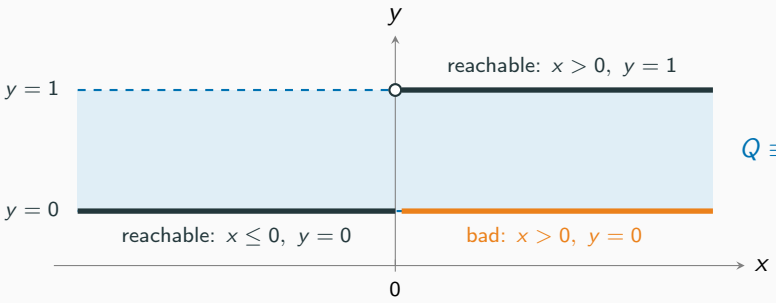
Next, `10 / y` needs $y \neq 0$, but $Q \wedge x > 0$ allows $y = 0$: division by zero?

What is happening: over-approximation



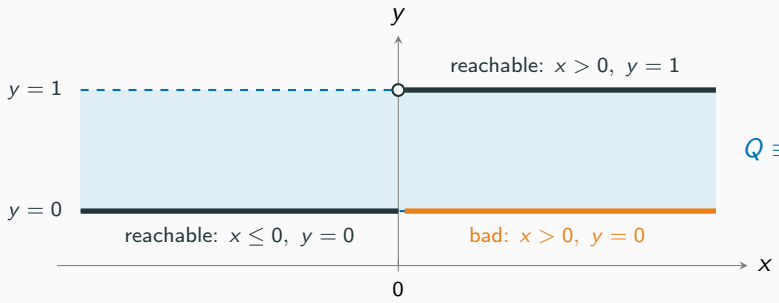
$Q \equiv 0 \leq y \leq 1$

What is happening: over-approximation



$Q \equiv 0 \leq y \leq 1$

What is happening: over-approximation



The bad states are inside Q, but no run reaches them.

Over-approximation: more states than reachable. Safe to prove, but alarms may be false.

Symbolic execution

Symbolic execution

```
if x > 0 then y := 1 else y := 0;
if x > 0 then z := 10 / y
```

Instead of one Q for both branches, follow the actual execution paths, one at a time:

execution path	state after first `if`	second `if`
$x > 0$	$y = 1$	$z := 10/1$ ✓
$x \leq 0$	$y = 0$	skipped ✓

Symbolic execution

```
if x > 0 then y := 1 else y := 0;
if x > 0 then z := 10 / y
```

Instead of one Q for both branches, follow the actual execution paths, one at a time:

execution path	state after first `if`	second `if`
$x > 0$	$y = 1$	$z := 10/1$ ✓
$x \leq 0$	$y = 0$	skipped ✓

Every state is reached by some input: no false alarm.

The mechanism for symbolic execution

Run the program on a symbolic input $x = \alpha$, not a number. Keep:

- a symbolic store σ: variable $\mapsto$ expression over the inputs;
- a path condition π: what the inputs must satisfy to get *here*.

The mechanism for symbolic execution

Run the program on a symbolic input $x = \alpha$, not a number. Keep:

- a symbolic store σ: variable $\mapsto$ expression over the inputs;
- a path condition π: what the inputs must satisfy to get *here*.

Example: one assignment, on symbolic input $y = \beta$.

$$\sigma = [\, y \mapsto \beta \,],\ \pi = \beta > 0 \quad \xrightarrow{y := y+1} \quad \sigma = [\, y \mapsto \beta + 1 \,],\ \pi = \beta > 0$$

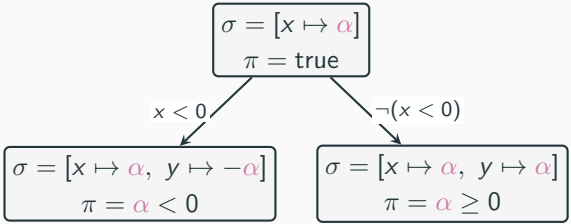
The execution tree

```
if x < 0 then
  y := -x
else
  y := x;
assert y > 0
```

$$\sigma = [x \mapsto \alpha]$$
$$\pi = \text{true}$$

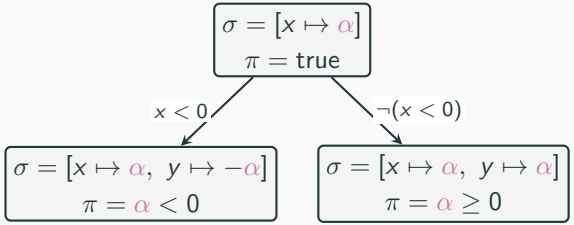
The execution tree

```
if x < 0 then
  y := -x
else
  y := x;
assert y > 0
```



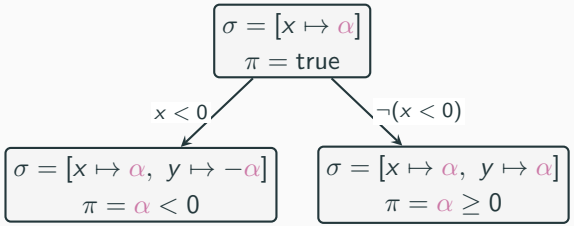
The execution tree

```
if x < 0 then
  y := -x
else
  y := x;
assert y > 0
```



The execution tree

```
if x < 0 then
  y := -x
else
  y := x;
assert y > 0
```



One leaf per path. Each leaf: a store and the condition to reach it.

Asking the solver

On each leaf: can the assertion fail? Is $\pi \wedge \neg(\sigma(y) > 0)$ satisfiable?

path	query	answer
left	$\alpha < 0 \ \wedge \ -\alpha \leq 0$	unsat: assertion holds

Asking the solver

On each leaf: can the assertion fail? Is $\pi \wedge \neg(\sigma(y) > 0)$ satisfiable?

path	query	answer
left	$\alpha < 0 \ \wedge \ -\alpha \leq 0$	unsat: assertion holds
right	$\alpha \geq 0 \ \wedge \ \alpha \leq 0$	sat, model $\alpha = 0$

Asking the solver

On each leaf: can the assertion fail? Is $\pi \wedge \neg(\sigma(y) > 0)$ satisfiable?

path	query	answer
left	$\alpha < 0 \ \wedge \ -\alpha \leq 0$	unsat: assertion holds
right	$\alpha \geq 0 \ \wedge \ \alpha \leq 0$	sat, model $\alpha = 0$

A test: `x = 0` gives `y = 0`, and the assertion fails.

Fork, don't merge

Hoare's If: one postcondition for both branches

$$\{\text{true}\}\ \texttt{if}\ \ldots\ \{y \geq 0\}$$

Contains $y = 0$, $x = 5$: not reachable.

Symbolic execution: one state per branch

$$\alpha < 0 \wedge y = -\alpha$$
$$\alpha \geq 0 \wedge y = \alpha$$

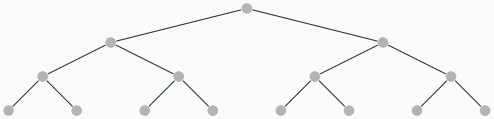
Every state is reachable.

Fork, don't merge

Hoare's If: one postcondition for both branches

$$\{\text{true}\}\ \texttt{if}\ \ldots\ \{y \geq 0\}$$

Contains $y = 0$, $x = 5$: not reachable.



Symbolic execution: one state per branch

$$\alpha < 0 \wedge y = -\alpha$$
$$\alpha \geq 0 \wedge y = \alpha$$

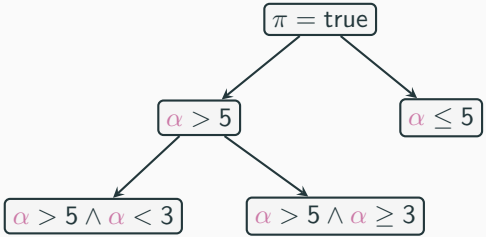
Every state is reachable.

n `ifs` $\Rightarrow$ up to 2^n paths

the price of path-sensitivity: path explosion

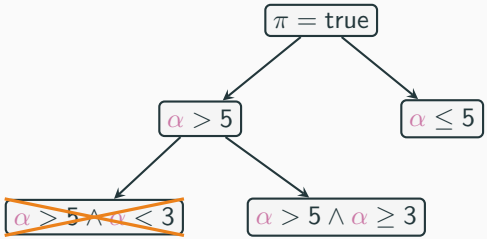
Infeasible paths

```
if x > 5 then
  if x < 3 then
    error
```



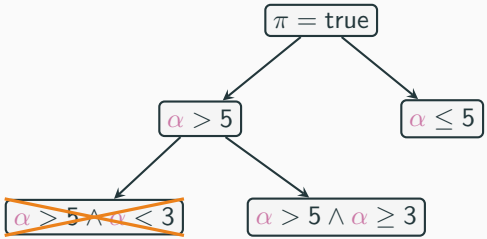
Infeasible paths

```
if x > 5 then
  if x < 3 then
    error
```



Infeasible paths

```
if x > 5 then
  if x < 3 then
    error
```



No input reaches `error`. At each fork, ask the solver: is π satisfiable? If not, drop the path.

What about loops?

Recap: a loop in Hoare logic

For $\{P\}\ c_0;\ \text{while } b \text{ do } c\ \{Q\}$ with invariant I:

$$\text{VC1}: P \Rightarrow \text{wp}(c_0, I) \qquad \text{VC2}: I \wedge b \Rightarrow \text{wp}(c, I) \qquad \text{VC3}: I \wedge \neg b \Rightarrow Q$$

$\{P \equiv \text{true}\}$	
$x := n;$	c_0
$\{I \equiv x \leq n\}$	VC1
while $x > 0$ do	$b \equiv x > 0$
$\{I \wedge b \equiv x \leq n \wedge x > 0\}$	
$x := x - 2$	c
$\{I \equiv x \leq n\}$	VC2
$\{I \wedge \neg b \equiv x \leq n \wedge \neg(x > 0)\}$	
$\{Q \equiv x \leq 0\}$	VC3

Recap: a loop in Hoare logic

For $\{P\}\ c_0;\ \text{while } b \text{ do } c\ \{Q\}$ with invariant I:

$$\text{VC1}: P \Rightarrow \text{wp}(c_0, I) \qquad \text{VC2}: I \wedge b \Rightarrow \text{wp}(c, I) \qquad \text{VC3}: I \wedge \neg b \Rightarrow Q$$

$\{P \equiv \text{true}\}$
  $x := n;$ — c_0
$\{I \equiv x \leq n\}$ — VC1
  while $x > 0$ do — $b \equiv x > 0$
    $\{I \wedge b \equiv x \leq n \wedge x > 0\}$
      $x := x - 2$ — c
    $\{I \equiv x \leq n\}$ — VC2
$\{I \wedge \neg b \equiv x \leq n \wedge \neg(x > 0)\}$
$\{Q \equiv x \leq 0\}$ — VC3

Verification conditions

VC1 init: $\text{true} \Rightarrow \text{wp}(x := n,\ x \leq n)$

VC2 preserve: $x \leq n \wedge x > 0 \Rightarrow \text{wp}(x := x - 2,\ x \leq n)$

VC3 exit: $x \leq n \wedge \neg(x > 0) \Rightarrow x \leq 0$

Proving VC1, VC2, VC3

Invariant $I \equiv x \leq n$.

VC1: $\quad \mathrm{wp}(x := n,\ x \leq n) = n \leq n \ \equiv\ \text{true},\ \ \text{so true} \Rightarrow \text{true}$ ✓

Proving VC1, VC2, VC3

Invariant $I \equiv x \leq n$.

VC1: $\mathrm{wp}(x := n,\ x \leq n) = n \leq n \ \equiv\ \text{true},$ so true $\Rightarrow$ true ✓

VC2: $\mathrm{wp}(x := x - 2,\ x \leq n) = x - 2 \leq n,$ and $x \leq n \wedge x > 0 \ \Rightarrow\ x - 2 \leq n$ ✓

Proving VC1, VC2, VC3

Invariant $I \equiv x \leq n$.

VC1: $\mathrm{wp}(x := n,\ x \leq n) = n \leq n \ \equiv\ \text{true},\ \text{so true} \Rightarrow \text{true}$ ✓

VC2: $\mathrm{wp}(x := x - 2,\ x \leq n) = x - 2 \leq n,\ \text{and } x \leq n \wedge x > 0 \ \Rightarrow\ x - 2 \leq n$ ✓

VC3: $x \leq n \wedge \neg(x > 0) \ \Rightarrow\ x \leq 0$ ✓

Proving VC1, VC2, VC3

Invariant $I \equiv x \leq n$.

VC1: $\mathrm{wp}(x := n,\ x \leq n) = n \leq n \ \equiv\ \text{true},$ so true $\Rightarrow$ true ✓

VC2: $\mathrm{wp}(x := x - 2,\ x \leq n) = x - 2 \leq n,$ and $x \leq n \wedge x > 0 \ \Rightarrow\ x - 2 \leq n$ ✓

VC3: $x \leq n \wedge \neg(x > 0) \ \Rightarrow\ x \leq 0$ ✓

So $\{\text{true}\}\ \ x := n;\ \text{while } x > 0 \text{ do } x := x - 2\ \ \{x \leq 0\}$.

Recap: what Hoare logic gives us

```
x := n;  while x > 0 do x := x - 2
```

On exit, Hoare logic gives $x \leq n \wedge x \leq 0$ (invariant in red).

Recap: what Hoare logic gives us

```
x := n;  while x > 0 do x := x - 2
```

On exit, Hoare logic gives $x \leq n \wedge x \leq 0$ (invariant in red).

For $n = 4$ this allows $x = -7$: not reachable. The only real exit value is $x = 0$.

Recap: what Hoare logic gives us

```
x := n;  while x > 0 do x := x - 2
```

On exit, Hoare logic gives $x \leq n \wedge x \leq 0$ (invariant in red).

For $n = 4$ this allows $x = -7$: not reachable. The only real exit value is $x = 0$.

The invariant covers every iteration, so it over-approximates.

Recap: what Hoare logic gives us

```
x := n;  while x > 0 do x := x - 2
```

On exit, Hoare logic gives $x \leq n \wedge x \leq 0$ (invariant in red).

For $n = 4$ this allows $x = -7$: not reachable. The only real exit value is $x = 0$.

The invariant covers every iteration, so it over-approximates.

How do we follow the actual runs of a loop instead?

Let's now use symbolic execution to find a bug

```
x := n;
while x > 0 do
  x := x - 2;
assert x = 0
```

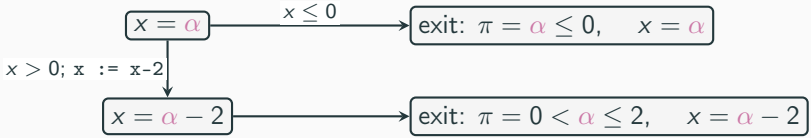
Input $n = \alpha$, symbolic. No invariant: unroll, forking at the guard each iteration.

Does the assertion ever fail?

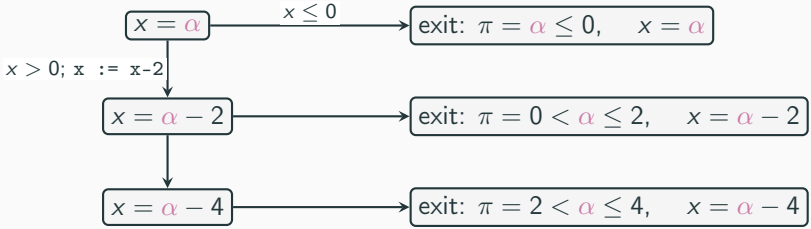
Unrolling, one iteration at a time



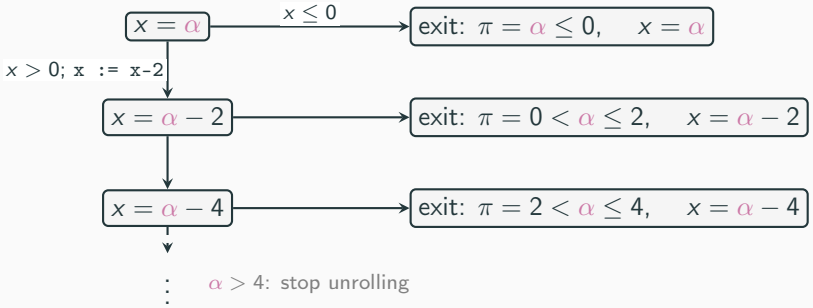
Unrolling, one iteration at a time



Unrolling, one iteration at a time



Unrolling, one iteration at a time



Asking the solver, again

On each exit: is $\pi \wedge \neg(\sigma(x) = 0)$ satisfiable?

query	answer	model
$\alpha \leq 0 \ \wedge \ \alpha \neq 0$	sat	$\alpha = -1$
$0 < \alpha \leq 2 \ \wedge \ \alpha - 2 \neq 0$	sat	$\alpha = 1$
$2 < \alpha \leq 4 \ \wedge \ \alpha - 4 \neq 0$	sat	$\alpha = 3$

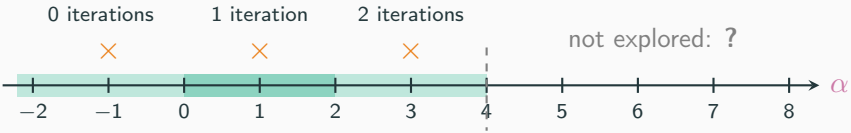
Asking the solver, again

On each exit: is $\pi \wedge \neg(\sigma(x) = 0)$ satisfiable?

query	answer	model
$\alpha \leq 0 \ \wedge \ \alpha \neq 0$	sat	$\alpha = -1$
$0 < \alpha \leq 2 \ \wedge \ \alpha - 2 \neq 0$	sat	$\alpha = 1$
$2 < \alpha \leq 4 \ \wedge \ \alpha - 4 \neq 0$	sat	$\alpha = 3$

Test `n = 1`: x ends at -1. The loop overshoots on odd input.

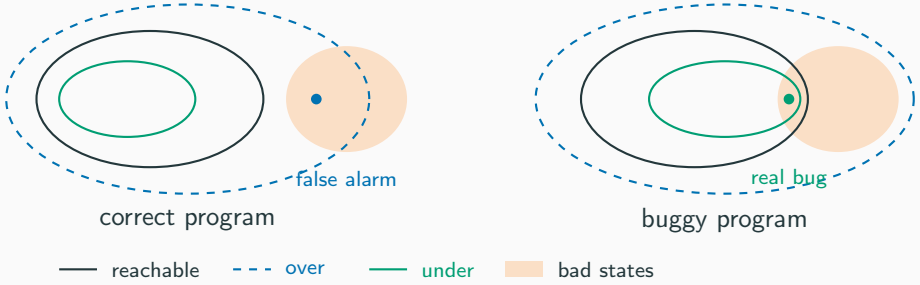
What we did *not* learn



- Every bug found is real: each comes with an input.
- About $\alpha > 4$: no claim at all. Silence is not a proof.

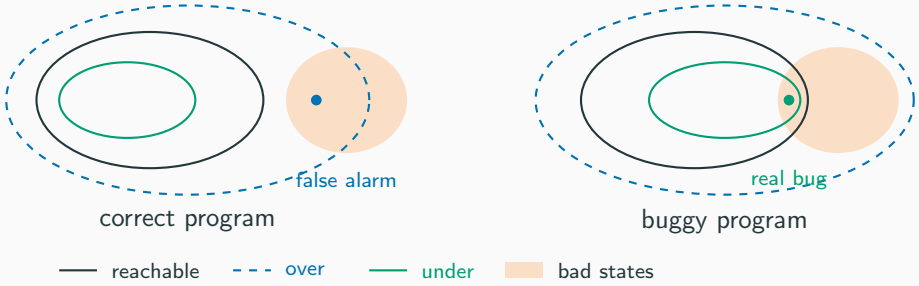
Which way to be wrong?

No analysis computes the reachable states exactly (undecidable). It must over- or under-shoot.



Which way to be wrong?

No analysis computes the reachable states exactly (undecidable). It must over- or under-shoot.



- Over (Hoare logic): silence *proves* no bug; an alarm may be false.
- Under (symbolic execution): an alarm is a *real* bug; silence proves nothing.

Symbolic execution in practice

tool	programs	known for
KLEE	C (LLVM)	test generation; bugs in GNU Coreutils
SAGE	Windows binaries	whitebox fuzzing of file parsers (Microsoft)
angr	binaries	binary analysis for security (Python)
Mayhem	binaries	won the DARPA Cyber Grand Challenge (2016)
Symbolic PathFinder	Java	NASA's symbolic extension of Java PathFinder
Pulse (in Infer)	C, C++, ...	under-approximate bug finding at Meta

Symbolic execution in practice

tool	programs	known for
KLEE	C (LLVM)	test generation; bugs in GNU Coreutils
SAGE	Windows binaries	whitebox fuzzing of file parsers (Microsoft)
angr	binaries	binary analysis for security (Python)
Mayhem	binaries	won the DARPA Cyber Grand Challenge (2016)
Symbolic PathFinder	Java	NASA's symbolic extension of Java PathFinder
Pulse (in Infer)	C, C++, ...	under-approximate bug finding at Meta

Many are concolic: run the program on a *concrete* input and *symbolically* at the same time, then flip a branch condition to get the next input.

Symbolic execution in practice

tool	programs	known for
KLEE	C (LLVM)	test generation; bugs in GNU Coreutils
SAGE	Windows binaries	whitebox fuzzing of file parsers (Microsoft)
angr	binaries	binary analysis for security (Python)
Mayhem	binaries	won the DARPA Cyber Grand Challenge (2016)
Symbolic PathFinder	Java	NASA's symbolic extension of Java PathFinder
Pulse (in Infer)	C, C++, ...	under-approximate bug finding at Meta

Many are concolic: run the program on a *concrete* input and *symbolically* at the same time, then flip a branch condition to get the next input.

Pulse: Infer's engine from Lecture 11, now under-approximating (incorrectness logic).

Summing up

Recap: symbolic execution

1. No invariants. An invariant over-approximates. So loops are unrolled, finitely.
2. No merging. Joining branches loses which states are real. So conditionals fork.
3. Witnesses. A state is real only if some input reaches it.

Recap: symbolic execution

1. No invariants. An invariant over-approximates. So loops are unrolled, finitely.
2. No merging. Joining branches loses which states are real. So conditionals fork.
3. Witnesses. A state is real only if some input reaches it.

Together: symbolic execution.

The whole picture

	Hoare / VCgen	**Symbolic execution**
approximation	over	under
silence means	bugs *proved* absent	nothing found *yet*
an alarm means	maybe a false alarm	a *genuine* bug
loop	invariant	unroll (bounded)
conditional	merge	fork
output	a proof	a test input

Today

- Over vs under: prove bugs absent, or report only real bugs.
- Under forces the mechanism: fork, unroll, and a satisfiability query that returns a test input.

Questions?

ATiPL 2026 Minute Sheet